

Create Hash Values

Introduction

In this activity, I practiced creating and comparing hash values using Linux commands. Hashing can be used to check whether a file has been changed or modified.

Scenario

I was given two files, `file1.txt` and `file2.txt`. When I displayed their contents using the `cat` command, they appeared to be the same. I needed to use SHA-256 hashing to determine whether the files were actually identical.

Task 1: Generate Hashes

First, I listed the files in the working directory:

```
ls
```

Then I displayed the contents of both files:

```
cat file1.txt  
cat file2.txt
```

Both files appeared to have the same contents.

I then generated a SHA-256 hash for each file:

```
sha256sum file1.txt  
sha256sum file2.txt
```

The hash values were different:

```
131f95c51cc819465fa1797f6ccacf9d494aaaff46fa3eac73ae63ffbd8267  file1.txt  
2558ba9a4cad1e69804ce03aa2a029526179a91a5e38cb723320e83af9ca017b  file2.txt
```

This showed that the files were not actually identical.

Task 2: Compare Hashes

I saved the hash values into separate files:

```
sha256sum file1.txt >> file1hash  
sha256sum file2.txt >> file2hash
```

Then I displayed the saved hashes:

```
cat file1hash
cat file2hash
```

Finally, I used `cmp` to compare the two hash files:

```
cmp file1hash file2hash
```

The result showed that the files differed at the first character.

Commands I Practiced

- `ls` — list files
- `cat` — display file contents
- `sha256sum` — generate a SHA-256 hash
- `cmp` — compare files

What I Learned

I learned that two files can appear identical when their contents are displayed, but their hash values can still show that they are different. Hashing is useful for checking the integrity of files and detecting unexpected changes.

Cybersecurity Connection

Hash values are useful to security analysts when verifying file integrity. A known trusted hash can be compared with a new hash to check whether a file has been modified.

Demonstration

Screenshot of the completed hashing and comparison tasks:

```
analyst@65dc93c4c2d2:~$ ls
file1.txt  file2.txt
analyst@65dc93c4c2d2:~$ cat file1.txt
X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
analyst@65dc93c4c2d2:~$ cat file2.txt
X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
9sxa5Yq20Ranalyst@65dc93c4c2d2:~$ sha256sum file1.txt
131f95c51cc819465fa1797f6ccacf9d494aaaff46fa3eac73ae63ffbdfd8267  file1.txt
analyst@65dc93c4c2d2:~$ sha256sum file2.txt
2558ba9a4cad1e69804ce03aa2a029526179a91a5e38cb723320e83af9ca017b  file2.txt
analyst@65dc93c4c2d2:~$ sha256sum file1.txt >> file1hash
analyst@65dc93c4c2d2:~$ sha256sum file2.txt >> file2hash
analyst@65dc93c4c2d2:~$ cat file1hash
131f95c51cc819465fa1797f6ccacf9d494aaaff46fa3eac73ae63ffbdfd8267  file1.txt
analyst@65dc93c4c2d2:~$
analyst@65dc93c4c2d2:~$ cat file2hash
2558ba9a4cad1e69804ce03aa2a029526179a91a5e38cb723320e83af9ca017b  file2.txt
analyst@65dc93c4c2d2:~$ cmp file1hash file2hash
file1hash file2hash differ: char 1, line 1
analyst@65dc93c4c2d2:~$
```

Conclusion

In this activity, I created SHA-256 hashes for two files and compared them using Linux commands. The different hash values showed that the files were not identical, even though their displayed contents appeared the same.